

Explain C++ vector line

1. explain this line in c++: // Create 2D vector to store value and weight // items[i][0] = value, items[i][1] = weight vector<vector<int>> items(n, vector<int>(2));	2
2. is a vector pair also a 2D vector? // https://www.geeksforgeeks.org/dsa/job-sequencing-problem/ // in decreasing order sort(jobs.begin(), jobs.end(), greater<pair<int, int>>());	5
3. what is this? // Sort items based on value-to-weight ratio in descending order sort(items.begin(), items.end(), compare);	8
4. can I use "greater" instead of "compare" here?	11
5. won't I need to call compare function as compare()?	14



1. explain this line in c++:

```
// Create 2D vector to store value and weight
// items[i][0] = value, items[i][1] = weight
vector<vector<int>> items(n, vector<int>(2));
```

Let's break down this C++ line step by step:

cpp

```
vector<vector<int>> items(n, vector<int>(2));
```

What it does

This line **creates a 2D vector** (a vector of vectors) named **items**.

It will look like a **table** with:

- **n** rows (one for each item)
- 2 columns (one for value, one for weight)

Step-by-step explanation

1. `vector<int>(2)`

- This creates a vector of **2 integers**.
- Since no values are given, both are initialized to **0**.
- Example: `vector<int>(2) → [0, 0]`

2. `vector<vector<int>> items(n, vector<int>(2));`

- This means:

Create a vector of size **n**, and each element is a vector of 2 integers.

- So the whole structure looks like:

```
cpp

items = [
    [0, 0],
    [0, 0],
    [0, 0],
    ...
];
```

(There are n rows in total.)

Usage meaning in context

From the comment:

```
cpp

// items[i][0] = value, items[i][1] = weight
```

It means:

- Each `items[i]` represents one item.
- `items[i][0]` stores the **value** of the item.
- `items[i][1]` stores the **weight** of the item.

Example

```
cpp

int n = 3;
vector<vector<int>> items(n, vector<int>(2));

// Assign values
items[0][0] = 60; // value
items[0][1] = 10; // weight

items[1][0] = 100;
items[1][1] = 20;

items[2][0] = 120;
items[2][1] = 30;
```

Now `items` looks like:

Item	Value	Weight
0	60	10

Item	Value	Weight
1	100	20
2	120	30

✓ Summary

`vector<vector<int>> items(n, vector<int>(2));`

→ creates a 2D vector (like an array of size $n \times 2$) initialized with zeros, used to store pairs of integers — here, **value** and **weight** for each item.



2. is a vector pair also a 2D vector?

```
// https://www.geeksforgeeks.org/dsa/job-sequencing-problem/
// in decreasing order
sort(jobs.begin(), jobs.end(),
     greater<pair<int, int>>());
```

Excellent question 🙌 — this touches on a subtle but important distinction in C++.

Let's unpack it carefully.

🧩 Short answer

A **vector of pairs** (`vector<pair<int, int>>`) is **not** the same as a **2D vector** (`vector<vector<int>>`).

They both can *store pairs of integers*, but their **structure**, **memory layout**, and **usage** differ.

🧱 Structural difference

Type	Meaning	Example element	Typical use
<code>vector<pair<int, int>></code>	A 1D vector where each element is a pair (2 linked values).	<code>{value, weight}</code>	When each pair of values belongs together logically (e.g., a job's profit & deadline).
<code>vector<vector<int>></code>	A 2D vector (vector of vectors) — more like a matrix or table .	<code>[60, 10]</code>	When you need a grid or table-like structure (e.g., DP matrix, or list of <code>[value, weight]</code> pairs).

🧩 Example difference

✅ 1. Vector of pairs:

cpp

```
vector<pair<int, int>> jobs = {
    {100, 2},
    {19, 1},
    {27, 2},
```

```
{25, 1},
{15, 3}
};
```

- Access:

```
cpp

jobs[i].first    // first value (e.g., profit)
jobs[i].second  // second value (e.g., deadline)
```

This is usually used when each item naturally consists of *two linked values*.

✓ 2. 2D Vector:

```
cpp

vector<vector<int>> items = {
    {100, 2},
    {19, 1},
    {27, 2}
};
```

- Access:

```
cpp

items[i][0]    // first value
items[i][1]    // second value
```

This feels more like a "table" of integers rather than a list of paired values.

⚙️ Now, about this line:

```
cpp

sort(jobs.begin(), jobs.end(), greater<pair<int, int>>());
```

Here:

- `jobs` is a `vector<pair<int,int>>`.
- `greater<pair<int,int>>()` sorts in **decreasing order**.

- The **comparison** is lexicographic — it compares `.first` first, then `.second` if the first is equal.

So if you had:

```
cpp

jobs = { {3, 4}, {2, 5}, {3, 2} };
```

After sorting with `greater<pair<int,int>>()`, you get:

```
{ {3, 4}, {3, 2}, {2, 5} }
```

Because $(3,4) > (3,2) > (2,5)$.

✓ Summary

Concept	Description
<code>vector<pair<int,int>></code>	1D vector — each element is a <i>pair</i> of related values.
<code>vector<vector<int>></code>	2D vector — each element is a <i>vector</i> (can have variable length).
<code>greater<pair<int,int>></code>	Comparator for sorting pairs in decreasing (descending) order.

👉 In short:

A *vector of pairs* is not a 2D vector — but both can store two integers per element. The difference is mainly **semantic** (pairs are linked, vectors are generic collections).



3. what is this?

```
// Sort items based on value-to-weight ratio in descending
order
sort(items.begin(), items.end(), compare);
```

Excellent — you're reading a **Knapsack-style sorting step**, which is quite common in greedy algorithms.

Let's fully break this line down 🙋

🧩 The line

cpp

```
sort(items.begin(), items.end(), compare);
```

🧱 What it does (in simple terms)

This line **sorts the items** in a vector, **using a custom comparison function** called `compare`. Specifically, as the comment says:

“Sort items based on value-to-weight ratio in descending order”

That means:

- Items with **higher value per unit weight** come **first**.
- It helps pick the most “profitable” item first (for problems like the **Fractional Knapsack**).

⚙️ Typical context

If `items` is defined like this:

cpp

```
vector<vector<int>> items = {
    {60, 10}, // value, weight
    {100, 20},
```



```
{120, 30}
};
```

then `items[i][0] = value`
and `items[i][1] = weight`.

The compare function

There's usually a comparator function defined earlier in the code like this:

```
cpp

bool compare(vector<int>& a, vector<int>& b) {
    double r1 = (double)a[0] / a[1]; // ratio of item a
    double r2 = (double)b[0] / b[1]; // ratio of item b
    return r1 > r2; // sort in descending order
}
```

So when we call:

```
cpp

sort(items.begin(), items.end(), compare);
```

it uses `compare()` every time it needs to decide which item should come before another.

Example in action

Before sorting:

```
cpp

items = [
    [60, 10], // ratio = 6.0
    [100, 20], // ratio = 5.0
    [120, 30] // ratio = 4.0
];
```

After sorting with `compare`:

```
cpp

items = [
    [60, 10], // ratio = 6.0
    [100, 20], // ratio = 5.0
    [120, 30] // ratio = 4.0
];
```

(Already sorted by ratio descending.)

 Summary

Component	Meaning
<code>sort(...)</code>	STL algorithm to sort a range
<code>items.begin(), items.end()</code>	defines the range (from start to end of vector)
<code>compare</code>	user-defined function that decides sorting order
Goal	Sorts items so those with higher value/weight ratio come first

Would you like me to show the **complete fractional knapsack example** where this sorting line is used?



4. can I use "greater" instead of "compare" here?

Excellent and very insightful question 🙌

Short answer:

❌ No, you **cannot** directly use `greater<>()` instead of `compare` here.

Let's see **why** — and when you *can*.

🧩 The current situation

Your code sorts:

```
cpp

sort(items.begin(), items.end(), compare);
```

Where each element of `items` is a vector like:

```
cpp

{value, weight}
```

And the goal of `compare` is:

```
cpp

bool compare(vector<int>& a, vector<int>& b) {
    double r1 = (double)a[0] / a[1]; // ratio for item a
    double r2 = (double)b[0] / b[1]; // ratio for item b
    return r1 > r2;                  // sort descending
}
```

⚙️ Why `greater<>` won't work

`greater<>()` is a **built-in comparator** that works only with types that have a defined **comparison operator (>)** — for example:

- integers

- floats
- strings
- `pair<int, int>` (because C++ defines how to compare pairs lexicographically)

But in your case:

- `items` is a `vector<vector<int>>`
- C++ has **no natural definition** for “greater” between two inner vectors (because: which one is “greater”? the first element? the sum? the ratio?)

So if you tried this:

```
cpp
sort(items.begin(), items.end(), greater<vector<int>>());
```

👉 It would cause a **compile-time error**, because `std::vector` does not have a built-in `>` comparison defined the way you need it.

When `greater<>` *can* be used

If your vector stores **pairs** (like `vector<pair<int, int>>`), you *can* use `greater<>()`:

```
cpp
vector<pair<int, int>> jobs;
sort(jobs.begin(), jobs.end(), greater<pair<int, int>>());
```

Because pairs have a default comparison rule:

- Compare `.first`, then `.second` if needed.

✅ Works fine.

But for `vector<vector<int>>`, you must define your **own comparator** (like `compare`) to control sorting logic — especially for custom criteria like *value/weight ratio*.

✅ Summary

Case	Example	Works with <code>greater<>?</code>	Why
<code>vector<int></code>	<code>sort(v.begin(), v.end(),</code>	✅	simple numeric

Case	Example	Works with greater<>?	Why
	<code>greater<int>());</code>		comparison
<code>vector<pair<int,int>></code>	<code>sort(v.begin(), v.end(), greater<pair<int,int>>());</code>	✓	pairs have lexicographic comparison
<code>vector<vector<int>></code>	<code>sort(v.begin(), v.end(), greater<vector<int>>());</code>	✗	no built-in "greater" for 2D vectors
Custom rule (like ratio)	<code>sort(v.begin(), v.end(), compare);</code>	✓	uses your custom logic

💡 In short:

Use `greater<>()` only for types that have a natural "greater than" meaning.

For complex structures (like 2D vectors or ratio-based sorting), **define your own comparator** like `compare`.



5. won't I need to call compare function as compare()?

Excellent catch — this is one of those subtle but fundamental C++ details that shows you're thinking like a real programmer 🙌

Let's unpack this carefully 📖

🧩 The question

You see this line:

```
cpp
sort(items.begin(), items.end(), compare);
```

and you wonder:

Shouldn't it be `compare()` instead of `compare`?

🧠 The key idea

No — you must **pass the function name without parentheses**.

✅ **Correct:**

```
cpp
sort(items.begin(), items.end(), compare);
```

❌ **Wrong:**

```
cpp
sort(items.begin(), items.end(), compare());
```

🧱 Why?

Let's see what's really happening.

`compare`

→ means the **function itself** (a *function pointer*).

You're giving the `sort()` algorithm the **address of your function**, so it can **call it repeatedly** while sorting.

`compare()`

→ means you are **calling** the function **immediately**, and passing its **return value** to `sort()`.

But that makes no sense here, because `compare()` returns a **bool** (true/false) — not a function.

And `sort()` expects a function (or something callable) that can compare two elements, not a single boolean value.

Example

Comparator function:

```
cpp

bool compare(vector<int>& a, vector<int>& b) {
    return (double)a[0] / a[1] > (double)b[0] / b[1];
}
```

When you do this:

```
cpp

sort(items.begin(), items.end(), compare);
```

C++ internally does something like:

```
cpp

if (compare(items[i], items[j])) { /* ... */ }
```

So `sort()` **calls** your function many times, comparing elements as it sorts.

If you wrote `compare()`, it would just run once **before** sorting, and give a single **true/false** — which doesn't make sense for sorting.

In summary

Expression	Meaning	Correct?
<code>compare</code>	Function pointer (the function itself)	✓
<code>compare()</code>	Calls the function immediately and passes the result	✗



Think of it like this:

You're *telling* sort "use this function", not "run this function right now."

Would you like me to also show the version where you use a **lambda function** instead of a separate `compare` function? (It's often shorter and cleaner.)